LÖVE WIKI · PART 2 OF 9

Guides

10 pages

wiki pages 1–10

Contents of this part

Main Page
love
Getting Started
Building LÖVE
Config Files
Game Distribution
License
Version History
Guidelines
Category:Tutorials

Main Page

Main Page · revision 27392 · last edited 2025-05-22

Welcome

As you probably know by now, LÖVE (http://love2d.org) is a framework for making 2D games in the Lua programming language. LÖVE is totally free, and can be used in anything from friendly open-source hobby projects, to evil, closed-source commercial ones.

Some interesting places to go from here:

- Getting Started
- Building LÖVE
- Tutorials
- love (The module)
- Game Distribution
- Config Files
- License (Free!)
- Games
- Libraries
- Software
- Snippets
- Version History

Lua

Never used Lua before? It's a really cool language! This manual won't teach you Lua, but fortunately there are other good resources for that.

- Programming in Lua (first edition) (http://lua.org/pil)
- Lua-Users Tutorials (http://lua-users.org/w iki/TutorialDirectory)
- Lua 5.1 Reference Manual (http://w w w.lua.org/manual/5.1/)

Hello World

This is the full source for 'hello world' in LÖVE. Running this code will cause an 800 by 600 window to appear, and display white text on a black background.

```
function love.draw()
    love.graphics.print('Hello World!', 400, 300)
end
```

Editing the wiki

Please read the Guidelines before editing the wiki!

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love

love · revision 31643 · last edited 2026-06-06

On this page

- Modules
- Third-party modules
- Functions
- Types
- Callbacks
  - General
- Window
- Keyboard
- Mouse
- Joystick
- Touch
- Other Languages

When writing games using LÖVE, the most important part of the API are the callbacks: love.load is for setting up the game state, love.update is for updating that state, and love.draw is for drawing the state to the screen.

Games may override additional callbacks in order to handle input from the user, handle errors, and more.

LÖVE provides blank placeholders for these callbacks, which you can override inside your own code by creating your own function with the same name as the callback:

```
-- Load some default values for our rectangle.
function love.load()
    x, y, w, h = 20, 20, 60, 20
end

-- Increase the size of the rectangle every frame.
function love.update(dt)
    w = w + 1
    h = h + 1
end

-- Draw a coloured rectangle.
function love.draw()
    -- In versions prior to 11.0, color component values are (0, 102, 102)
    love.graphics.setColor(0, 0.4, 0.4)
    love.graphics.rectangle("fill", x, y, w, h)
end
```

Modules

love.audio	Provides of audio interface for playback/recording sound.	
love.data	Provides functionality for creating and transforming data.	Added since 11.0
love.event	Manages events, like keypresses.	Added since 0.6.0
love.filesystem	Provides an interface to the user's filesystem.	
love.font	Allows you to work with fonts.	Added since 0.7.0
love.graphics	Drawing of shapes and images, management of screen geometry.	
love.image	Provides an interface to decode encoded image data.	
love.joystick	Provides an interface to connected joysticks.	Added since 0.5.0
love.keyboard	Provides an interface to the user's keyboard.	
love.math	Provides system-independent mathematical functions.	Added since 0.9.0
love.mouse	Provides an interface to the user's mouse.	
love.physics	Can simulate 2D rigid body physics in a realistic manner.	Added since 0.4.0
love.sound	This module is responsible for decoding sound files.	
love.system	Provides access to information about the user's system.	Added since 0.9.0

love.thread	Allows you to work with threads.	Added since 0.7.0
love.timer	Provides high-resolution timing functionality.	
love.touch	Provides an interface to touch-screen presses.	Added since 0.10.0
love.video	This module is responsible for decoding and streaming video files.	Added since 0.10.0
love.window	Provides an interface for the program's window.	Added since 0.9.0

Third-party modules

lua-enet	Multiplayer networking module for games.	Added since 0.9.0
socket	Module for HTTP, TCP, and UDP networking.	Added since 0.5.0
utf8	Provides basic support for manipulating UTF-8 strings.	Added since 0.9.2

Functions

love.getVersion	Gets the current running version of LÖVE.	Added since 0.9.1
love.hasDeprecationOutput	Gets whether LÖVE displays warnings when using deprecated functionality.	Added since 11.0
love.isVersionCompatible	Gets whether the given version is compatible with the current running version of LÖVE.	Added since 0.10.0
love.setDeprecationOutput	Sets whether LÖVE displays warnings when using deprecated functionality.	Added since 11.0

Types

Data	The superclass of all data.
Object	The superclass of all LÖVE types.
Variant	The types supported by love.thread and love.event.

Callbacks

All callbacks are only called in main thread.

General

Config Files	Game configuration settings.		
love.draw	Callback function used to draw on the screen every frame.		
love.errhand	The error handler, used to display error messages.		Deprecated in 11.0
love.errorhandler	The error handler, used to display error messages.	Added since 11.0	
love.load	This function is called exactly once at the beginning of the game.		
love.lowmemory	Callback function triggered when the system is running out of memory on mobile devices.	Added since 0.10.0	
love.quit	Callback function triggered when the game is closed.	Added since 0.7.0	
love.run	The main callback function, containing the main loop. A sensible default is used when left out.		
love.threaderror	Callback function triggered when a Thread encounters an error.	Added since 0.9.0	

love.update	Callback function used to update the state of the game every frame.

Window

love.directorydropped	Callback function triggered when a directory is dragged and dropped onto the window.	Added since 0.10.0
love.displayrotated	Called when the device display orientation changed.	Added since 11.3
love.filedropped	Callback function triggered when a file is dragged and dropped onto the window.	Added since 0.10.0
love.focus	Callback function triggered when window receives or loses focus.	Added since 0.7.0
love.mousefocus	Callback function triggered when window receives or loses mouse focus.	Added since 0.9.0
love.resize	Called when the window is resized.	Added since 0.9.0
love.visible	Callback function triggered when window is shown or hidden.	Added since 0.9.0

Keyboard

love.keypressed	Callback function triggered when a key is pressed.	
love.keyreleased	Callback function triggered when a keyboard key is released.	
love.textedited	Called when the candidate text for an IME has changed.	Added since 0.10.0
love.textinput	Called when text has been entered by the user.	Added since 0.9.0

Mouse

love.mousemoved	Callback function triggered when the mouse is moved.	Added since 0.9.2
love.mousepressed	Callback function triggered when a mouse button is pressed.	
love.mousereleased	Callback function triggered when a mouse button is released.	
love.wheelmoved	Callback function triggered when the mouse wheel is moved.	Added since 0.10.0

Joystick

love.gamepadaxis	Called when a Joystick's virtual gamepad axis is moved.	Added since 0.9.0
love.gamepadpressed	Called when a Joystick's virtual gamepad button is pressed.	Added since 0.9.0
love.gamepadreleased	Called when a Joystick's virtual gamepad button is released.	Added since 0.9.0
love.joystickadded	Called when a Joystick is connected.	Added since 0.9.0
love.joystickaxis	Called when a joystick axis moves.	Added since 0.9.0
love.joystickhat	Called when a joystick hat direction changes.	Added since 0.9.0
love.joystickpressed	Called when a joystick button is pressed.	
love.joystickreleased	Called when a joystick button is released.	
love.joystickremoved	Called when a Joystick is disconnected.	Added since 0.9.0

Touch

love.touchmoved	Callback function triggered when a touch press moves inside the touch screen.	Added since 0.10.0
love.touchpressed	Callback function triggered when the touch screen is touched.	Added since 0.10.0
love.touchreleased	Callback function triggered when the touch screen stops being touched.	Added since 0.10.0

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Getting Started

Getting Started · revision 29650 · last edited 2025-08-10

On this page

- Get LÖVE
- Making a Game
- Running Games
  - Windows
  - Linux
  - macOS
  - Android
  - iOS
- Next steps
- Other Languages

Get LÖVE

Download the latest version of LÖVE from the website (http://love2d.org/#download), and install it. If you're on Windows and don't want to *install* LÖVE, you can also just download the zipped executables and extract them anywhere.

To find out which version of LÖVE is installed, run the following command:

```
love --version
```

Note that you may need to add the directory where LÖVE resides to your system's PATH variable, in order to be able to run it from the command line.

Making a Game

To make a minimal game, create a folder anywhere, and open up your favorite text editor. Sublime Text (http://www.sublimetext.com) is a pretty good one for all operating systems, and it has Lua support built in. Create a new file in the folder you just created, and name it `main.lua`. Put the following code in the file, and save it.

```
function love.draw()
    love.graphics.print("Hello World", 400, 300)
end
```

Running Games

LÖVE can load a game in two ways:

- From a folder that contains a main.lua file.
- From a .love file that has a main.lua file in the top-most directory level (aka root)

For creating .love files see Game Distribution.

Windows

ZeroBrane Studio (http://studio.zerobrane.com), Sublime Text, Notepad++, and SciTE allow you to launch the game from within their code editors.

Otherwise, the easiest way to run the game is to drag the folder onto either love.exe or a shortcut to love.exe. Remember to drag the folder containing `main.lua`, and not `main.lua` itself.

You can also launch the game from the command line:

```
"C:\Program Files\LOVE\love.exe" "C:\games\mygame"
"C:\Program Files\LOVE\love.exe" "C:\games\packagedgame.love"
```

You can create a shortcut to do this; simply make a shortcut to love.exe, right-click on it and select "Properties", and then put the command line you want in the "Target" box for the shortcut.

On Windows, there is a special command-line option which will attach a console to the window, allowing you to see the result of `print` calls (equivalent to setting `t.console=true` in conf.lua or running `lovec.exe` (since 0.10.2)):

```
"C:\Program Files\LOVE\love.exe" --console
```

Linux

On Linux, you can use one of these command lines:

```
love /home/path/to/gamedir/
love /home/path/to/packagedgame.love
```

If you installed LÖVE system-wide, you can double click on .love files in your file manager as well.

---

macOS

On macOS, a folder or .love file can be dropped onto the love application bundle. On the Mac Terminal (command line), you can use love like this (assuming it's installed to the Applications directory):

```
open -n -a love "~/path/to/mygame"
```

However, the above method will not output printed text to the terminal window. To do that, you will need to execute the love binary inside the application bundle directly:

```
/Applications/love.app/Contents/MacOS/love ~/path/to/mygame
```

You can set up an alias in your Terminal session to call the binary when you use `love` by adding an alias to your `~/.zshrc` file (Z shell configuration file).

Open the file with

```
open -a TextEdit ~/.zshrc
```

You may have to run

```
touch ~/.zshrc
```

first if the file does not yet exist.

Then paste in the following code and save the file:

```
# alias to love
alias love="/Applications/love.app/Contents/MacOS/love"
```

Now you can call love from the command line like Linux and Windows:

```
love ~/path/to/mygame
```

if this doesn't works you should reload the .zshrc file like this:

```
source ~/.zshrc
```

and try it again.

Android

As long as you have LÖVE for Android APK sideloaded from the official site, the following methods will work:

- Transfer your game folder to `/sdcard/Android/data/org.love2d.android/files/games/lovegame` (`/sdcard/lovegame` also works in Pie and earlier) where `main.lua` is found at `/sdcard/Android/data/org.love2d.android/files/games/lovegame/main.lua`. In Android 11 or later, these directories only accessible through MTP in PC. Then run LÖVE for Android.
- Transfer/download a .love of your game to the device and click on it. Some file explorers will run the .love using LÖVE for Android. If it does not you can install ES File Explorer which will. If you use Dropbox, you can use that to open .love files. If that doesn't work, use LÖVE Loader and pick the .love file there.

If you have more questions, consider reading Android-specific FAQ (https://github.com/love2d/love-android/wiki/FAQ---Frequently-Asked-Questions). If your question is not listed, then feel free to ask in the forums or the Discord.

---

iOS

In order to run LÖVE for iOS, it must first be compiled and installed. To do that, you'll need macOS, Xcode 7 or newer (https://developer.apple.com/xcode/), and the LÖVE for iOS source code downloadable from the home page (https://love2d.org).

- If the `libraries` folder isn't present in the `love/platform/xcode/ios` folder, download the iOS libraries from the homepage (https://love2d.org/), and place the `iOS/libraries` folder from the zip there. They contain the third-party library dependencies used by LÖVE.
- Open the Xcode project found at `love/platform/xcode/love.xcodeproj` and select the `love-ios` target in the dropdown menu at the top of the window.
- You may want to change the Build Configuration from Debug to Release for better performance, by opening the "Edit Scheme..." menu from the same dropdown selection.
- Choose either an iOS Simulator device or your plugged-in iOS device in the dropdown selection to the right of the previous one, and click the Build-and-Run ▶ button to the left, which will install LÖVE on the target device after compiling it.

LÖVE on iOS includes a simple list interface of games that are installed (until you fuse a .love to it for distribution.)

- To put a .love file on the iOS Simulator after LÖVE is installed, drag the file onto the iOS Simulator's window while it's open. LÖVE will launch if it's not running already. If another game is currently active you

may need to quit LÖVE for the new game to show up (press Shift-Command-H twice to open the App Switcher menu on the iOS Simulator.)

- To put a .love file or game folder on your iOS device after LÖVE is installed, you can either download it with the Safari, or transfer it from your computer through iTunes when your device is connected: open iTunes, go to the iOS device which has LÖVE installed, go to the 'Apps' section and scroll down and find LÖVE, and add the .love file or game folder to LÖVE's Documents section. On more recent iOS and Mac versions, you can use Airdrop to transfer your .love file from your Mac to where LÖVE is installed.

See the Game Distribution page for creating Fused LÖVE games on iOS and distributing them.

Note: for those interested in creating a LÖVE game directly on iOS, the Textastic app (available on the Apple App Store for purchase) can utilize SublimeText's packages-- one of which is a LÖVE package. You can follow these instructions to load that LÖVE package into the Textastic app on your iOS device to author LÖVE code directly on your iOS device. You'll then need to follow the earlier instructions on how to zip up that code into a .love file but then can either export that .love file to a LÖVE app running on your iOS device or an external device running a LÖVE application.

Instructions:

1. Purchase the iOS Textastic app on the App Store
2. Create a folder in Textastic (while inside the Textastic app "Local Files" section or in the iOS Files app under the Textastic folder) called "#Textastic"
3. go to https://github.com/szensk/subllualove ↗ in Safari on your iPad
4. tap the green "Code" button and select "Download ZIP"
5. Unzip the file in the Files app. This will create a folder called "subllualove-master"
6. Move or copy that folder into the folder "On my iPad/Textastic/#Textastic/". This is equivalent to this folder in Textastic: "Local Files/#Textastic/".
7. In Textastic, go to Settings (gear wheel on the bottom left portion of the app) -> Other and tap "Reload Customizations"
8. In File Properties (document looking icon on top right portion of the app) -> Syntax Definition, there is now a new item "LOVE (*.lua)" that you can select after opening a .lua file.

If you have any questions on this process, please contact the Textastic developer (very responsive by email support)

Next steps

- Tutorial:Callback_Functions will teach you the basic structure of a love game.
- Category:Tutorials are the next piece of reading.
- Wiki The LÖVE wiki.

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Building LÖVE

Building LÖVE · revision 29932 · last edited 2025-09-30

On this page

1. Getting the source
2. Dependencies
   - Windows
   - Mac OS X
   - Linux
3. Building
   - Windows
   - Mac OS X
   - Linux

Other Languages


This page is still incomplete, you are likely to be eaten by a grue.


This page has information on compiling the LÖVE source code. This is only necessary if you want to make modifications or additions to the framework itself, or if a binary is not provided for your platform (for example, GNU/Linux distributions other than Ubuntu or 32-bit GNU/Linux). If you just want to code a game in Lua, go back to Getting Started.


LÖVE for Android has its own build instructions which deviates entirely from this page. See the repository (https://github.com/love2d/love-android) for more information!

Still interested? Okay. Follow these three steps and you'll have your very own LÖVE binary:

1. Getting the source
2. Dependencies
3. Building

1. Getting the source

The official repository of LÖVE is hosted on GitHub (https://github.com/love2d/love).

You can download the source code either by downloading an archive from one of the tags on the download page (https://github.com/love2d/love/releases) or by installing Git and cloning the repository. Almost every Linux distributions have Git in their repositories, some even have it installed by default. Windows and OS X users can get the official binary of Git from here (https://git-scm.com/downloads).

To create a clone with Git, open a terminal or command prompt, go to the directory that you want to download the source code folder to and use the following commands:

```
git clone https://github.com/love2d/love
```

This will create a love folder and download the latest source code into it.

If you want to use the source code of a specific version, go into the love directory and issue one of the following commands:

```
git checkout 0.8.0      # any (released) version from 0.6.0 onwards.
git checkout 0.7.2
```

```
git checkout 11.x       # to the latest commit at stable branch (currently 11.x)
git checkout main       # to the latest commit at upcoming version 12.0
```

2. Dependencies

LÖVE uses several open source libraries such as Lua, libvorbis, etc. Some are shipped alongside LÖVE source code but some must be installed manually. Those libraries and their development files are required to successfully build LÖVE. It's possible to build every dependency yourself, but this is out of the scope of this page. The following explains how you can get prebuilt binaries instead.

Windows

The dependencies and instructions for LÖVE 0.9.0+ can be found here (https://github.com/love2d/megasource).

Mac OS X

The required pre-built frameworks are available here (https://love2d.org/sdk/). Place the frameworks in `/Library/Frameworks/` in order for LÖVE's Xcode project to recognize them.

Linux

Download the development packages of the dependencies from the repository of your distribution.

Note on LÖVE 12.0

LÖVE 12 requires **SDL3** instead of SDL2, which may not provided by your distro. We recommend using love-appimage-source (https://github.com/love2d/love-appimage-source) to compile LÖVE 12. If for any reason you cannot use it, you must have SDL3 development files installed.

LÖVE 12 no longer uses mpg123. It is safe to compile LÖVE 12 without it present.

Ubuntu and Debian Jessie onwards

```
sudo apt-get install build-essential autotools-dev automake libtool pkg-config \
libfreetype6-dev libluajit-5.1-dev libsdl3-dev libopenal-dev libogg-dev \
libvorbis-dev libmodplug-dev libtheora-dev libharfbuzz-dev
```

These are taken from the `platform/unix/debian/control.in` file in the repository.

For the 0.9.0 branch, older versions of Ubuntu need the latest libsdl2-dev packages, which can be found via this ppa: https://launchpad.net/~bartbes/+archive/love-stable ↗

If you get an error during the build complaining about libturbojpeg, you must first fix a bug in Ubuntu, then run `configure` again:

```
ln -s /usr/lib/x86_64-linux-gnu/libturbojpeg.so.0 /usr/lib/x86_64-linux-gnu/libturbojpeg.so
```

Notes:

- Prior to 10.0, `libdevil` is also needed.
- Prior to 11.0, `libphysfs-dev` is also needed.
- Prior to 12.0, `libmpg123-dev` is also needed.

- Starting on 12.0, `libsdl3-dev` is required, which is not available in any stable channels. 11.x requires `libsdl2-dev` instead.

Arch Linux

```
pacman -S devil flac freetype2 glibc libmodplug libvorbis lua51 mesa mpg123 openal physfs sdl # 0.6.x
pacman -S devil freetype2 libmodplug libvorbis lua51 mpg123 openal physfs sdl # 0.7.x
pacman -S luajit physfs freetype2 devil mpg123 openal libvorbis libmodplug sdl2 shared-mime-info hicolor-icon-theme desktop-file-utils # for 0.9.x (taken from package in repository)
pacman -S luajit freetype2 mpg123 openal libvorbis libtheora libmodplug sdl2 shared-mime-info hicolor-icon-theme desktop-file-utils # for 11.x
pacman -S luajit freetype2 openal libvorbis libtheora libmodplug sdl3 shared-mime-info hicolor-icon-theme desktop-file-utils # for 12.x, see note below
```

Fedora

```
dnf install freetype-devel glibc-devel libmpg123-devel libmodplug-devel physfs-devel mesa-libGL-devel openal-soft-devel libvorbis-devel SDL-devel libmng-devel libtiff-devel lua-devel gcc-c++ libtool  luajit luajit-devel libtheora-devel SDL2-devel
```

Please note that libmpg123-devel is in the rpmfusion-free repository.

Mageia 2

```
urpmi make automake autoconf libopenal-devel libtool libtool-base libphysfs-devel libSDL-devel lua liblua-devel libmodplug-devel libfreetype6-devel libmpg123-devel libvorbis-devel libmng-devel libxpm-devel libxcursor-devel libxxf86vm-devel
```

Gentoo

```
emerge -va lua physfs libsdl opengl freetype openal libvorbis mpg123 libmodplug
```

Make sure you have autotools installed (which should normally be the case):

```
emerge -va automake autoconf libtool
```

Nix/NixOS

NixOS provides it's own official build script (https://github.com/NixOS/nixpkgs/blob/master/pkgs/development/interpreters/love/11.nix).

Other distributions

Any other distribution will most likely have the dependencies in their repositories too, possibly with slightly different package names.

3. Building

Windows

Follow the instructions at the megasource (https://github.com/love2d/megasource) repository page to build LÖVE 0.9.0+.

Be sure to keep the required DLLs in the same folder as love.exe when running the executable.

Mac OS X

Open the `love.xcodeproj` file with Xcode and press 'build', or use the `xcodebuild` command-line utility.

Linux

Use the automagic script to generate the configure file and then run configure and make as usual.

Be aware that if you got your source from the LÖVE homepage (https://love2d.org/) your configure script has been pre-generated. This means that the ./platform/unix/automagic step should be skipped.

```
./platform/unix/automagic
./configure
make
```

Your LÖVE binary will end up in the src directory.

Note: If you need to build an AppImage, check love-appimage-source (https://github.com/love2d/love-appimage-source) repository instead.

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Config Files

Config Files · revision 30474 · last edited 2025-12-03

On this page

- Introduction
- love.conf
- Current Configuration File
- Flags
- Older Versions
- See Also
- Other Languages

Introduction

If a file called `conf.lua` is present in your game folder (or .love file), it is run *before* the LÖVE modules are loaded. You can use this file to overwrite the `love.conf` function, which is later called by the LÖVE 'boot' script. Using the `love.conf` function, you can set some configuration options, and change things such as the default size of the window, which modules are loaded, and other program setup-related options.

`love.conf` must be defined inside a file named `conf.lua`. It will not work if it's defined inside `main.lua`, because it affects initialization that happens before `main.lua` is loaded.

love.conf

The `love.conf` function takes one argument: a table filled with all the default values which you can overwrite to your liking. If you want to change the default window size, for instance, do:

```
function love.conf(t)
    t.window.width = 1024
    t.window.height = 768
end
```

If you don't need the physics module or joystick module, do the following.

```
function love.conf(t)
    t.modules.joystick = false
    t.modules.physics = false
end
```

Setting unused modules to false is encouraged when you release your game. It reduces startup time slightly (especially if the joystick module is disabled) and reduces memory usage (slightly).

Note that you can't disable love.filesystem and love.data; it's mandatory. The same goes for the love module itself. love.graphics needs love.window to be enabled.

In LÖVE version 0.9.2 and earlier, errors in the config file will cause the game to not launch and no error message to appear. If the game doesn't load, check the config file for errors first. In version 0.10.2 and later, errors in the config will now show the blue error screen showing the error in the config file.

Current Configuration File

Here is a full list of options and their default values for LÖVE 11.3 and 11.4:

```
function love.conf(t)
    t.identity = nil                    -- The name of the save directory (string)
```

```
    t.appendidentity = false            -- Search files in source directory before save directory (boolean)
    t.version = "11.5"                  -- The LÖVE version this game was made for (string)
    t.console = false                   -- Attach a console (boolean, Windows only)
    t.accelerometerjoystick = true      -- Enable the accelerometer on iOS and Android by exposing it as a
Joystick (boolean)
    t.externalstorage = false           -- True to save files (and read from the save directory) in external
storage on Android (boolean)
    t.gammacorrect = false              -- Enable gamma-correct rendering, when supported by the system (boolean)

    t.audio.mic = false                 -- Request and use microphone capabilities in Android (boolean)
    t.audio.mixwithsystem = true        -- Keep background music playing when opening LOVE (boolean, iOS and
Android only)

    t.window.title = "Untitled"         -- The window title (string)
    t.window.icon = nil                 -- Filepath to an image to use as the window's icon (string)
    t.window.width = 800                -- The window width (number)
    t.window.height = 600               -- The window height (number)
    t.window.borderless = false         -- Remove all border visuals from the window (boolean)
    t.window.resizable = false          -- Let the window be user-resizable (boolean)
    t.window.minwidth = 1               -- Minimum window width if the window is resizable (number)
    t.window.minheight = 1              -- Minimum window height if the window is resizable (number)
    t.window.fullscreen = false         -- Enable fullscreen (boolean)
    t.window.fullscreentype = "desktop" -- Choose between "desktop" fullscreen or "exclusive" fullscreen mode
(string)
    t.window.vsync = 1                  -- Vertical sync mode (number)
    t.window.msaa = 0                   -- The number of samples to use with multi-sampled antialiasing (number)
    t.window.depth = nil                -- The number of bits per sample in the depth buffer
    t.window.stencil = nil              -- The number of bits per sample in the stencil buffer
    t.window.display = 1                -- Index of the monitor to show the window in (number)
    t.window.highdpi = false            -- Enable high-dpi mode for the window on a Retina display (boolean)
    t.window.usedpiscale = true         -- Enable automatic DPI scaling when highdpi is set to true as well
(boolean)
    t.window.x = nil                    -- The x-coordinate of the window's position in the specified display
(number)
    t.window.y = nil                    -- The y-coordinate of the window's position in the specified display
(number)

    t.modules.audio = true              -- Enable the audio module (boolean)
    t.modules.data = true               -- Enable the data module (boolean)
    t.modules.event = true              -- Enable the event module (boolean)
    t.modules.font = true               -- Enable the font module (boolean)
    t.modules.graphics = true           -- Enable the graphics module (boolean)
    t.modules.image = true              -- Enable the image module (boolean)
    t.modules.joystick = true           -- Enable the joystick module (boolean)
    t.modules.keyboard = true           -- Enable the keyboard module (boolean)
    t.modules.math = true               -- Enable the math module (boolean)
    t.modules.mouse = true              -- Enable the mouse module (boolean)
    t.modules.physics = true            -- Enable the physics module (boolean)
    t.modules.sound = true              -- Enable the sound module (boolean)
    t.modules.system = true             -- Enable the system module (boolean)
    t.modules.thread = true             -- Enable the thread module (boolean)
    t.modules.timer = true              -- Enable the timer module (boolean), Disabling it will result 0 delta
time in love.update
    t.modules.touch = true              -- Enable the touch module (boolean)
    t.modules.video = true              -- Enable the video module (boolean)
    t.modules.window = true             -- Enable the window module (boolean)
end
```

Flags

identity

This flag determines the name of the save directory for your game. Note that you can only specify the name, not the location where it will be created:

```
t.identity = "gabe_HL3" -- Correct
t.identity = "c:/Users/gabe/HL3" -- Incorrect
```

Alternatively love.filesystem.setIdentity can be used to set the save directory outside of the config file.

appendidentity

Available since LÖVE 11.0

This flag is not supported in earlier versions.

This flag determines if game directory should be searched first then save directory (`true`) or otherwise (`false`)

version

Available since LÖVE 0.8.0

This flag is not supported in earlier versions.

`t.version` should be a string, representing the version of LÖVE for which your game was made.

Before 11.0, it should be formatted as "*X*.*Y*.*Z*" where *X* is the major release number, *Y* the minor, and *Z* the patch level. Since 11.0, it should be formatted as "*X*.*Y*" where *X* and *Y* are the major and minor release respectively.

If set in the config file of the game, LÖVE will display a warning if the game isn't compatible with the current version of LÖVE being used to run the game. Its default is the version of LÖVE running.

console

Determines whether a console should be opened alongside the game window (Windows only) or not. Note: On OSX you can get console output by running LÖVE through the terminal, or on Windows with LÖVE 0.10.2, by running `lovec.exe` instead of `love.exe`.

accelerometerjoystick

Available since LÖVE 0.10.0

This flag is not supported in earlier versions.

Sets whether the device accelerometer on iOS and Android should be exposed as a 3-axis Joystick. Disabling the accelerometer when it's not used may reduce CPU usage.

externalstorage

Available since LÖVE 0.10.1

This flag is not supported in earlier versions.

Sets whether files are saved in external storage (true) or internal storage (false) on Android.

gammacorrect

Available since LÖVE 0.10.0

This flag is not supported in earlier versions.

Determines whether gamma-correct rendering is enabled, when the system supports it.

audio.mic

Available since LÖVE 11.3

This flag is not supported in earlier versions.

Request microphone permission from the user. When user allows it, love.audio.getRecordingDevices will lists recording devices available. Otherwise, love.audio.getRecordingDevices returns empty table and a message is shown to inform user when called.

audio.mixwithsystem

Available since LÖVE 11.0

This flag is not supported in earlier versions.

Sets whether background audio / music from other apps should play while LÖVE is open. See love.system.hasBackgroundMusic for more details.

window

Available since LÖVE 0.9.0

These flags are not supported in earlier versions.

It is possible to defer window creation until love.window.setMode is first called in your code. To do so, set `t.window = nil` in love.conf (or `t.screen = nil` in older versions.) If this is done, LÖVE may crash if any function from love.graphics is called before the first love.window.setMode in your code.

The `t.window` table was named `t.screen` in versions prior to 0.9.0. The `t.screen` table doesn't exist in love.conf in 0.9.0, and the `t.window` table doesn't exist in love.conf in 0.8.0. This means **love.conf** will fail to execute (therefore it will fall back to default values) if care is not taken to use the correct table for the LÖVE version being used.

window.title

Available since LÖVE 0.9.0

This flag is not supported in earlier versions.

Sets the title of the window the game is in. Alternatively love.window.setTitle can be used to change the window title outside of the config file.

window.icon

Available since LÖVE 0.9.0

This flag is not supported in earlier versions.

A filepath to an image to use as the window's icon. Not all operating systems support very large icon images. The icon can also be changed with love.window.setIcon.

window.width & window.height

Available since LÖVE 0.9.0

These flags are not supported in earlier versions.

Sets the window's dimensions. If these flags are set to 0 LÖVE automatically uses the user's desktop dimensions. In mobile, the aspect ratio will be used to determine if the game runs in landscape or portrait.

window.borderless

Available since LÖVE 0.9.0

This flag is not supported in earlier versions.

Removes all border visuals from the window. Note that the effects may vary between operating systems.

window.resizable

Available since LÖVE 0.9.0

This flag is not supported in earlier versions.

If set to true this allows the user to resize the game's window. In version 11.4 and later for Android, this also allows changing orientation between landscape and portrait.

window.minwidth & window.minheight

Available since LÖVE 0.9.0

These flags are not supported in earlier versions.

Sets the minimum width and height for the game's window if it can be resized by the user. If you set lower values to `window.width` and `window.height` LÖVE will always favor the minimum dimensions set via `window.minwidth` and `window.minheight`.

window.fullscreen

Available since LÖVE 0.9.0

This flag is not supported in earlier versions.

Wether to run the game in fullscreen (`true`) or windowed (`false`) mode. The fullscreen can also be toggled via love.window.setFullscreen or love.window.setMode. In 11.3 for Android, setting this to `true` hides the status bar.

window.fullscreentype

Available since LÖVE 0.9.0

This flag is not supported in earlier versions.

Specifies the type of fullscreen mode to use (`exclusive` or `desktop`). Generally the `desktop` is recommended, as it is less restrictive than `exclusive` mode on some operating systems. (Note: In 0.9.2 and earlier, use `normal` instead of `exclusive`.)

window.vsync

Available since LÖVE 0.9.0

This flag is not supported in earlier versions.

Enables or deactivates vertical synchronization. Vsync tries to keep the game at a steady framerate and can prevent issues like screen tearing. It is recommended to keep vsync activated if you don't know about the possible implications of turning it off. Before LÖVE 11.0, this value was boolean (`true` or `false`). Since LÖVE 11.0, this value is a number (`-1` to use adaptive vsync when supported, `0` to disable vsync, `1` to enable vsync; `2` or a larger number will wait that many frames before refreshing - this is driver-dependent, however.)

Note that in iOS, vertical synchronization is always enabled and cannot be changed.

window.depth

Available since LÖVE 11.0

This flag is not supported in earlier versions.

The number of bits per sample in the depth buffer (16/24/32, default `nil`)

window.stencil

Available since LÖVE 11.0

This flag is not supported in earlier versions.

Then number of bits per sample in the stencil buffer (generally 8, default `nil`)

window.msaa

Available since LÖVE 0.9.2

This flag is not supported in earlier versions.

The number of samples to use with multi-sampled antialiasing.

window.display

Available since LÖVE 0.9.0

This flag is not supported in earlier versions.

The index of the display to show the window in, if multiple monitors are available.

window.highdpi

Available since LÖVE 0.9.1

This flag is not supported in earlier versions.

See love.window.getDPIScale. Allows the window's backbuffer to use the full pixel density of high-DPI displays on supported operating systems. LOVE will automatically scale things accordingly and use DPI-scaled units instead of pixels for most things (since 11.0) when this is true, unless the `usedpiscale` flag is set to false. When `highdpi` is false, the OS will keep things consistent between low-DPI and high-DPI displays by rendering to a low-resolution backbuffer and scaling that up on its own, when a high-DPI display is used.

This flag currently does nothing on Windows, and on Android it's effectively always enabled.

window.usedpiscale

Available since LÖVE 11.3

This flag is not supported in earlier versions.

Disables LOVE's automatic DPI scaling on high-DPI displays when false. This only has an effect when the `highdpi` flag is set to true, since the OS (rather than LOVE) takes care of everything otherwise.

window.x & window.y

Available since LÖVE 0.9.2

These flags are not supported in earlier versions.

Determines the position of the window on the user's screen. Alternatively love.window.setPosition can be used to change the position on the fly.

window.fsaa

Available since LÖVE 0.9.0 and removed in LÖVE 0.10.0

This flag has been replaced by the window.msaa flag.

The number of samples to use with multi-sampled antialiasing.

window.srgb

Available since LÖVE 0.9.1 and removed in LÖVE 0.10.0

This flag has been replaced by the gammacorrect flag.

Enabling this window flag will automatically convert the colors of everything drawn to the main screen from the linear RGB colorspace to the sRGB colorspace - the window's surface is treated as gamma-space sRGB. This is only one component of gamma-correct rendering, an advanced topic which is easy to mess up, so it's recommended to keep this option disabled if you're not sure about its implications.

Release Mode

Available since LÖVE 0.8.0 and removed in LÖVE 0.9.0

This flag is not supported in earlier or later versions.

If `t.release` is enabled, LÖVE uses the release error handler, which is sparse on information by default, and can, of course, be overridden.

The default release mode error handler also outputs a message to the player informing them to contact the author using the values *title, author and url* as specified in conf.lua.

When a fused game in release mode is run it will not save in the love save dir, but rather one for itself, whereas previously it would be %APPDATA%\\LOVE\\game on Windows, it now is %APPDATA%\\game. This concept applies to other platforms as well.

Older Versions

Here is a full list of options and their default values for LÖVE 11.0 until 11.2:

```
function love.conf(t)
    t.identity = nil                    -- The name of the save directory (string)
    t.appendidentity = false            -- Search files in source directory before save directory (boolean)
    t.version = "11.0"                  -- The LÖVE version this game was made for (string)
    t.console = false                   -- Attach a console (boolean, Windows only)
    t.accelerometerjoystick = true      -- Enable the accelerometer on iOS and Android by exposing it as a
Joystick (boolean)
    t.externalstorage = false           -- True to save files (and read from the save directory) in external
storage on Android (boolean)
    t.gammacorrect = false              -- Enable gamma-correct rendering, when supported by the system (boolean)

    t.audio.mixwithsystem = true        -- Keep background music playing when opening LOVE (boolean, iOS and
Android only)

    t.window.title = "Untitled"         -- The window title (string)
    t.window.icon = nil                 -- Filepath to an image to use as the window's icon (string)
    t.window.width = 800                -- The window width (number)
    t.window.height = 600               -- The window height (number)
```

```
    t.window.borderless = false         -- Remove all border visuals from the window (boolean)
    t.window.resizable = false          -- Let the window be user-resizable (boolean)
    t.window.minwidth = 1               -- Minimum window width if the window is resizable (number)
    t.window.minheight = 1              -- Minimum window height if the window is resizable (number)
    t.window.fullscreen = false         -- Enable fullscreen (boolean)
    t.window.fullscreentype = "desktop" -- Choose between "desktop" fullscreen or "exclusive" fullscreen mode
(string)
    t.window.vsync = 1                  -- Vertical sync mode (number)
    t.window.msaa = 0                   -- The number of samples to use with multi-sampled antialiasing (number)
    t.window.depth = nil                -- The number of bits per sample in the depth buffer
    t.window.stencil = nil              -- The number of bits per sample in the stencil buffer
    t.window.display = 1                -- Index of the monitor to show the window in (number)
    t.window.highdpi = false            -- Enable high-dpi mode for the window on a Retina display (boolean)
    t.window.x = nil                    -- The x-coordinate of the window's position in the specified display
(number)
    t.window.y = nil                    -- The y-coordinate of the window's position in the specified display
(number)

    t.modules.audio = true              -- Enable the audio module (boolean)
    t.modules.data = true               -- Enable the data module (boolean)
    t.modules.event = true              -- Enable the event module (boolean)
    t.modules.font = true               -- Enable the font module (boolean)
    t.modules.graphics = true           -- Enable the graphics module (boolean)
    t.modules.image = true              -- Enable the image module (boolean)
    t.modules.joystick = true           -- Enable the joystick module (boolean)
    t.modules.keyboard = true           -- Enable the keyboard module (boolean)
    t.modules.math = true               -- Enable the math module (boolean)
    t.modules.mouse = true              -- Enable the mouse module (boolean)
    t.modules.physics = true            -- Enable the physics module (boolean)
    t.modules.sound = true              -- Enable the sound module (boolean)
    t.modules.system = true             -- Enable the system module (boolean)
    t.modules.thread = true             -- Enable the thread module (boolean)
    t.modules.timer = true              -- Enable the timer module (boolean), Disabling it will result 0 delta
time in love.update
    t.modules.touch = true              -- Enable the touch module (boolean)
    t.modules.video = true              -- Enable the video module (boolean)
    t.modules.window = true             -- Enable the window module (boolean)
end
```

Here is a full list of options and their default values for LÖVE 0.10.1 and 0.10.2:

```
function love.conf(t)
    t.identity = nil                    -- The name of the save directory (string)
    t.version = "0.10.2"                -- The LÖVE version this game was made for (string)
    t.console = false                   -- Attach a console (boolean, Windows only)
    t.accelerometerjoystick = true      -- Enable the accelerometer on iOS and Android by exposing it as a
Joystick (boolean)
    t.externalstorage = false           -- True to save files (and read from the save directory) in external
storage on Android (boolean)
    t.gammacorrect = false              -- Enable gamma-correct rendering, when supported by the system (boolean)

    t.window.title = "Untitled"         -- The window title (string)
    t.window.icon = nil                 -- Filepath to an image to use as the window's icon (string)
    t.window.width = 800                -- The window width (number)
    t.window.height = 600               -- The window height (number)
    t.window.borderless = false         -- Remove all border visuals from the window (boolean)
    t.window.resizable = false          -- Let the window be user-resizable (boolean)
    t.window.minwidth = 1               -- Minimum window width if the window is resizable (number)
```

```
    t.window.minheight = 1              -- Minimum window height if the window is resizable (number)
    t.window.fullscreen = false         -- Enable fullscreen (boolean)
    t.window.fullscreentype = "desktop" -- Choose between "desktop" fullscreen or "exclusive" fullscreen mode
(string)
    t.window.vsync = true               -- Enable vertical sync (boolean)
    t.window.msaa = 0                   -- The number of samples to use with multi-sampled antialiasing (number)
    t.window.display = 1                -- Index of the monitor to show the window in (number)
    t.window.highdpi = false            -- Enable high-dpi mode for the window on a Retina display (boolean)
    t.window.x = nil                    -- The x-coordinate of the window's position in the specified display
(number)
    t.window.y = nil                    -- The y-coordinate of the window's position in the specified display
(number)

    t.modules.audio = true              -- Enable the audio module (boolean)
    t.modules.event = true              -- Enable the event module (boolean)
    t.modules.graphics = true           -- Enable the graphics module (boolean)
    t.modules.image = true              -- Enable the image module (boolean)
    t.modules.joystick = true           -- Enable the joystick module (boolean)
    t.modules.keyboard = true           -- Enable the keyboard module (boolean)
    t.modules.math = true               -- Enable the math module (boolean)
    t.modules.mouse = true              -- Enable the mouse module (boolean)
    t.modules.physics = true            -- Enable the physics module (boolean)
    t.modules.sound = true              -- Enable the sound module (boolean)
    t.modules.system = true             -- Enable the system module (boolean)
    t.modules.timer = true              -- Enable the timer module (boolean), Disabling it will result 0 delta
time in love.update
    t.modules.touch = true              -- Enable the touch module (boolean)
    t.modules.video = true              -- Enable the video module (boolean)
    t.modules.window = true             -- Enable the window module (boolean)
    t.modules.thread = true             -- Enable the thread module (boolean)
end
```

Here is a full list of options and their default values for LÖVE 0.10.0:

```
function love.conf(t)
    t.identity = nil                    -- The name of the save directory (string)
    t.version = "0.10.0"                -- The LÖVE version this game was made for (string)
    t.console = false                   -- Attach a console (boolean, Windows only)
    t.accelerometerjoystick = true      -- Enable the accelerometer on iOS and Android by exposing it as a
Joystick (boolean)
    t.gammacorrect = false              -- Enable gamma-correct rendering, when supported by the system (boolean)

    t.window.title = "Untitled"         -- The window title (string)
    t.window.icon = nil                 -- Filepath to an image to use as the window's icon (string)
    t.window.width = 800                -- The window width (number)
    t.window.height = 600               -- The window height (number)
    t.window.borderless = false         -- Remove all border visuals from the window (boolean)
    t.window.resizable = false          -- Let the window be user-resizable (boolean)
    t.window.minwidth = 1               -- Minimum window width if the window is resizable (number)
    t.window.minheight = 1              -- Minimum window height if the window is resizable (number)
    t.window.fullscreen = false         -- Enable fullscreen (boolean)
    t.window.fullscreentype = "desktop" -- Choose between "desktop" fullscreen or "exclusive" fullscreen mode
(string)
    t.window.vsync = true               -- Enable vertical sync (boolean)
    t.window.msaa = 0                   -- The number of samples to use with multi-sampled antialiasing (number)
    t.window.display = 1                -- Index of the monitor to show the window in (number)
    t.window.highdpi = false            -- Enable high-dpi mode for the window on a Retina display (boolean)
    t.window.x = nil                    -- The x-coordinate of the window's position in the specified display
```

```
(number)
    t.window.y = nil                    -- The y-coordinate of the window's position in the specified display
(number)

    t.modules.audio = true              -- Enable the audio module (boolean)
    t.modules.event = true              -- Enable the event module (boolean)
    t.modules.graphics = true           -- Enable the graphics module (boolean)
    t.modules.image = true              -- Enable the image module (boolean)
    t.modules.joystick = true           -- Enable the joystick module (boolean)
    t.modules.keyboard = true           -- Enable the keyboard module (boolean)
    t.modules.math = true               -- Enable the math module (boolean)
    t.modules.mouse = true              -- Enable the mouse module (boolean)
    t.modules.physics = true            -- Enable the physics module (boolean)
    t.modules.sound = true              -- Enable the sound module (boolean)
    t.modules.system = true             -- Enable the system module (boolean)
    t.modules.timer = true              -- Enable the timer module (boolean), Disabling it will result 0 delta
time in love.update
    t.modules.touch = true              -- Enable the touch module (boolean)
    t.modules.video = true              -- Enable the video module (boolean)
    t.modules.window = true             -- Enable the window module (boolean)
    t.modules.thread = true             -- Enable the thread module (boolean)
end
```

Here is a full list of options and their default values for LÖVE 0.9.2:

```
function love.conf(t)
    t.identity = nil                    -- The name of the save directory (string)
    t.version = "0.9.2"                 -- The LÖVE version this game was made for (string)
    t.console = false                   -- Attach a console (boolean, Windows only)

    t.window.title = "Untitled"         -- The window title (string)
    t.window.icon = nil                 -- Filepath to an image to use as the window's icon (string)
    t.window.width = 800                -- The window width (number)
    t.window.height = 600               -- The window height (number)
    t.window.borderless = false         -- Remove all border visuals from the window (boolean)
    t.window.resizable = false          -- Let the window be user-resizable (boolean)
    t.window.minwidth = 1               -- Minimum window width if the window is resizable (number)
    t.window.minheight = 1              -- Minimum window height if the window is resizable (number)
    t.window.fullscreen = false         -- Enable fullscreen (boolean)
    t.window.fullscreentype = "normal" -- Choose between "normal" fullscreen or "desktop" fullscreen mode
(string)
    t.window.vsync = true               -- Enable vertical sync (boolean)
    t.window.fsaa = 0                   -- The number of samples to use with multi-sampled antialiasing (number)
    t.window.display = 1                -- Index of the monitor to show the window in (number)
    t.window.highdpi = false            -- Enable high-dpi mode for the window on a Retina display (boolean)
    t.window.srgb = false               -- Enable sRGB gamma correction when drawing to the screen (boolean)
    t.window.x = nil                    -- The x-coordinate of the window's position in the specified display
(number)
    t.window.y = nil                    -- The y-coordinate of the window's position in the specified display
(number)

    t.modules.audio = true              -- Enable the audio module (boolean)
    t.modules.event = true              -- Enable the event module (boolean)
    t.modules.graphics = true           -- Enable the graphics module (boolean)
    t.modules.image = true              -- Enable the image module (boolean)
    t.modules.joystick = true           -- Enable the joystick module (boolean)
    t.modules.keyboard = true           -- Enable the keyboard module (boolean)
    t.modules.math = true               -- Enable the math module (boolean)
```

```
    t.modules.mouse = true              -- Enable the mouse module (boolean)
    t.modules.physics = true            -- Enable the physics module (boolean)
    t.modules.sound = true              -- Enable the sound module (boolean)
    t.modules.system = true             -- Enable the system module (boolean)
    t.modules.timer = true              -- Enable the timer module (boolean), Disabling it will result 0 delta
time in love.update
    t.modules.window = true             -- Enable the window module (boolean)
    t.modules.thread = true             -- Enable the thread module (boolean)
end
```

Here is a full list of options and their default values for LÖVE 0.9.1:

```
function love.conf(t)
    t.identity = nil                    -- The name of the save directory (string)
    t.version = "0.9.1"                 -- The LÖVE version this game was made for (string)
    t.console = false                   -- Attach a console (boolean, Windows only)

    t.window.title = "Untitled"         -- The window title (string)
    t.window.icon = nil                 -- Filepath to an image to use as the window's icon (string)
    t.window.width = 800                -- The window width (number)
    t.window.height = 600               -- The window height (number)
    t.window.borderless = false         -- Remove all border visuals from the window (boolean)
    t.window.resizable = false          -- Let the window be user-resizable (boolean)
    t.window.minwidth = 1               -- Minimum window width if the window is resizable (number)
    t.window.minheight = 1              -- Minimum window height if the window is resizable (number)
    t.window.fullscreen = false         -- Enable fullscreen (boolean)
    t.window.fullscreentype = "normal" -- Standard fullscreen or desktop fullscreen mode (string)
    t.window.vsync = true               -- Enable vertical sync (boolean)
    t.window.fsaa = 0                   -- The number of samples to use with multi-sampled antialiasing (number)
    t.window.display = 1                -- Index of the monitor to show the window in (number)
    t.window.highdpi = false            -- Enable high-dpi mode for the window on a Retina display (boolean)
    t.window.srgb = false               -- Enable sRGB gamma correction when drawing to the screen (boolean)

    t.modules.audio = true              -- Enable the audio module (boolean)
    t.modules.event = true              -- Enable the event module (boolean)
    t.modules.graphics = true           -- Enable the graphics module (boolean)
    t.modules.image = true              -- Enable the image module (boolean)
    t.modules.joystick = true           -- Enable the joystick module (boolean)
    t.modules.keyboard = true           -- Enable the keyboard module (boolean)
    t.modules.math = true               -- Enable the math module (boolean)
    t.modules.mouse = true              -- Enable the mouse module (boolean)
    t.modules.physics = true            -- Enable the physics module (boolean)
    t.modules.sound = true              -- Enable the sound module (boolean)
    t.modules.system = true             -- Enable the system module (boolean)
    t.modules.timer = true              -- Enable the timer module (boolean)
    t.modules.window = true             -- Enable the window module (boolean)
    t.modules.thread = true             -- Enable the thread module (boolean)
end
```

Here is a full list of options and their default values for LÖVE 0.9.0:

```
function love.conf(t)
    t.identity = nil                    -- The name of the save directory (string)
    t.version = "0.9.0"                 -- The LÖVE version this game was made for (string)
    t.console = false                   -- Attach a console (boolean, Windows only)

    t.window.title = "Untitled"         -- The window title (string)
    t.window.icon = nil                 -- Filepath to an image to use as the window's icon (string)
```

```
    t.window.width = 800               -- The window width (number)
    t.window.height = 600              -- The window height (number)
    t.window.borderless = false        -- Remove all border visuals from the window (boolean)
    t.window.resizable = false         -- Let the window be user-resizable (boolean)
    t.window.minwidth = 1              -- Minimum window width if the window is resizable (number)
    t.window.minheight = 1             -- Minimum window height if the window is resizable (number)
    t.window.fullscreen = false        -- Enable fullscreen (boolean)
    t.window.fullscreentype = "normal" -- Standard fullscreen or desktop fullscreen mode (string)
    t.window.vsync = true              -- Enable vertical sync (boolean)
    t.window.fsaa = 0                  -- The number of samples to use with multi-sampled antialiasing (number)
    t.window.display = 1               -- Index of the monitor to show the window in (number)

    t.modules.audio = true             -- Enable the audio module (boolean)
    t.modules.event = true             -- Enable the event module (boolean)
    t.modules.graphics = true          -- Enable the graphics module (boolean)
    t.modules.image = true             -- Enable the image module (boolean)
    t.modules.joystick = true          -- Enable the joystick module (boolean)
    t.modules.keyboard = true          -- Enable the keyboard module (boolean)
    t.modules.math = true              -- Enable the math module (boolean)
    t.modules.mouse = true             -- Enable the mouse module (boolean)
    t.modules.physics = true           -- Enable the physics module (boolean)
    t.modules.sound = true             -- Enable the sound module (boolean)
    t.modules.system = true            -- Enable the system module (boolean)
    t.modules.timer = true             -- Enable the timer module (boolean)
    t.modules.window = true            -- Enable the window module (boolean)
    t.modules.thread = true            -- Enable the thread module (boolean)
end
```

Here is a full list of options and their default values for LÖVE 0.8.0:

```
function love.conf(t)
    t.title = "Untitled"        -- The title of the window the game is in (string)
    t.author = "Unnamed"        -- The author of the game (string)
    t.url = nil                 -- The website of the game (string)
    t.identity = nil            -- The name of the save directory (string)
    t.version = "0.8.0"         -- The LÖVE version this game was made for (string)
    t.console = false           -- Attach a console (boolean, Windows only)
    t.release = false           -- Enable release mode (boolean)
    t.screen.width = 800        -- The window width (number)
    t.screen.height = 600       -- The window height (number)
    t.screen.fullscreen = false -- Enable fullscreen (boolean)
    t.screen.vsync = true       -- Enable vertical sync (boolean)
    t.screen.fsaa = 0           -- The number of MSAA samples (number)
    t.modules.joystick = true   -- Enable the joystick module (boolean)
    t.modules.audio = true      -- Enable the audio module (boolean)
    t.modules.keyboard = true   -- Enable the keyboard module (boolean)
    t.modules.event = true      -- Enable the event module (boolean)
    t.modules.image = true      -- Enable the image module (boolean)
    t.modules.graphics = true   -- Enable the graphics module (boolean)
    t.modules.timer = true      -- Enable the timer module (boolean)
    t.modules.mouse = true      -- Enable the mouse module (boolean)
    t.modules.sound = true      -- Enable the sound module (boolean)
    t.modules.physics = true    -- Enable the physics module (boolean)
    t.modules.thread = true     -- Enable the thread module (boolean)
end
```

Here is a full list of options and their default values for LÖVE 0.7.2 and earlier:

```lua
function love.conf(t)
    t.title = "Untitled"        -- The title of the window the game is in (string)
    t.author = "Unnamed"        -- The author of the game (string)
    t.identity = nil            -- The name of the save directory (string)
    t.version = 0               -- The LÖVE version this game was made for (number)
    t.console = false           -- Attach a console (boolean, Windows only)
    t.screen.width = 800        -- The window width (number)
    t.screen.height = 600       -- The window height (number)
    t.screen.fullscreen = false -- Enable fullscreen (boolean)
    t.screen.vsync = true       -- Enable vertical sync (boolean)
    t.screen.fsaa = 0           -- The number of MSAA samples (number)
    t.modules.joystick = true   -- Enable the joystick module (boolean)
    t.modules.audio = true      -- Enable the audio module (boolean)
    t.modules.keyboard = true   -- Enable the keyboard module (boolean)
    t.modules.event = true      -- Enable the event module (boolean)
    t.modules.image = true      -- Enable the image module (boolean)
    t.modules.graphics = true   -- Enable the graphics module (boolean)
    t.modules.timer = true      -- Enable the timer module (boolean)
    t.modules.mouse = true      -- Enable the mouse module (boolean)
    t.modules.sound = true      -- Enable the sound module (boolean)
    t.modules.physics = true    -- Enable the physics module (boolean)
end
```

See Also

- love

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Game Distribution

Game Distribution · revision 30619 · last edited 2026-02-28

On this page

- Create a .love-file
  - Platform Specific Notes
- Creating a Windows Executable
  - Platform Specific Instructions
- Creating a macOS Application
- Distribution for Linux
- Android
  - Compiling from source
  - Using APKTool
- iOS
- Distribution on the web
- Community Tools
- Other Languages

Once you have completed your game you surely want to make it available to others. In general you should first create a .love-file which can be either directly distributed (which requires users to have LÖVE installed) or used for platform specific packages.

These are the preferred ways to distribute your game depending on target (all make use of the .love file):

- Windows → Windows Executable
- macOS → macOS Application
- Linux → plain .love file, or Appimages

Create a .love-file

Please note that some operating systems use case-sensitive paths. To avoid unnecessary hassle make sure that the path specifications you use in code matches that of your folders and files.

To create a .love-file you have to create a zip file of the whole game directory. Make sure that your main.lua is in the root of the archive, e.g. if you have

```
<somedir>\SuperGame\gfx\character.png
<somedir>\SuperGame\main.lua
<somedir>\SuperGame\conf.lua
```

then you must ensure the zip file contains

```
gfx\character.png
main.lua
conf.lua
```

and **not** the directory `SuperGame\`.

Once you have your zip file you simply rename the ending from .zip to .love and you are done!

Note: If you're using a case-insensitive file system (e.g. Windows and macOS), try running your game using the generated .love file. This will reveal any case sensitivity problems that other users may run into when trying to run your game. A common pitfall is to have filename with different capitalization than it's in the game code because once the game is zipped, it becomes case-sensitive.

Platform Specific Notes

Windows

Windows supports creation of zip files out of the box via right-click menu. For detailed instructions see Compress and uncompress files (zip files) (https://support.microsoft.com/en-us/help/14200/windows-compress-uncompress-zip-files).

macOS

Inside the game folder, select the game files and folders, right-click/Ctrl+click and pick **Compress n items**. Then rename the resulting `.zip` file to have a `.love` ending.

Alternatively, you can use the terminal. Navigate inside the game folder and run:

```
zip -9 -r SuperGame.love .
```

Linux

Assuming your current directory is `SuperGame/` you can create the .love file from command line directly using

```
zip -9 -r SuperGame.love .
```

Creating a Windows Executable

Once you have packed your game into a .love file you can create a game executable that directly runs your game.

For this you have to append your .love file to the love.exe file that comes with the official LÖVE .zip file. The resulting file is your game executable.

Once you have your game executable you can pack it together with all the other DLL files of the official LÖVE .zip file into a new .zip file and share this with the world.

Platform Specific Instructions

Windows

To create your game executable under windows use the command line to run

```
copy /b love.exe+SuperGame.love SuperGame.exe
```

Note: You may have to run the command as an administrator on your computer. You can do this by going to **Windows\System32** and right clicking on **cmd.exe** and selecting the option **Run as administrator**. After this, you must type **cd C:\Program Files\LOVE** (or whatever your default love directory is), then you may continue as normal.

Alternatively you can create a .bat (e.g. create_game_exe.bat) file with the contents

```
copy /b love.exe+%1 "%~n1.exe"
```

and then you can simply drag your SuperGame.love file onto the .bat file and it will create the file SuperGame.love.exe which you could then use for distribution.

If you are using **PowerShell** instead of cmd, then use the following command:

```
Get-Content love.exe,SuperGame.love -Encoding Byte | Set-Content SuperGame.exe -Encoding Byte
```

or invoke cmd from **PowerShell** using the following:

```
cmd /c copy /b love.exe+SuperGame.love SuperGame.exe
```

This method creates a **fused** game.

When distributing the Windows executable you will need to include the dll files that came in the folder of the love.exe you used (so don't mix the 32 bit dll's with the 64 bit dll's). Without these files you will get error message when attempting to run SuperGame.exe on a windows machine.

The contents of your final distribution folder should look something like this:

- SDL2.dll
- OpenAL32.dll (note: this file is different in the 64 bit download despite still being called 'OpenAL32.dll')
- SuperGame.exe
- license.txt (note: the license **requires** that it be included in any further distribution)
- love.dll
- lua51.dll
- mpg123.dll
- msvcp120.dll
- msvcr120.dll

Windows icon

You can use your own icon images using tools like for example the freeware Resource Hacker (http://www.angusj.com/resourcehacker/).

Video tutorial (https://youtu.be/WTk_HJdXVNQ?t=2m22s) and step-by-step instructions (http://gamedev.stackexchange.com/a/121947/9093) are available.

Linux / macOS

You can create the windows executable from Linux and macOS using the command line.

First download the official zipped 32 or 64 bit executable (**not** the installer) from https://www.love2d.org/ (https://www.love2d.org/). 32 bit games can run on 64 bit Windows systems but not vice versa, so to save time on smaller games, you can provide 32bit only.

Then in the command line/terminal run

```
cat love.exe SuperGame.love > SuperGame.exe
```

to obtain the game executable `SuperGame.exe`.

Creating a macOS Application

For official/commercial distribution (e.g., in the Mac App Store) skip to the next section, Official Distribution and the App Store.

Once you have your game prepared as .love file you can make your game available for macOS users by following the steps below (video tutorial (https://youtu.be/SU2RpGdezP4) available):

1. Download and extract the Mac version of LÖVE from the LÖVE homepage (https://www.love2d.org)
2. Rename `love.app` to `SuperGame.app`
3. Copy your `SuperGame.love` to `SuperGame.app/Contents/Resources/` (right-click/Ctrl+click and pick "Show Package Content" on macOS). This will result in the game running in fused mode.
4. Modify `SuperGame.app/Contents/Info.plist` (see below for details)
5. Zip the `SuperGame.app` folder (e.g. to SuperGame_osx.zip) and distribute it. Enable the *-y* flag of zip to keep the symlinks.

When modifying `SuperGame.app/Contents/Info.plist` make sure to change the string values of the following XML-tags:

- `CFBundleIdentifier`
- `CFBundleName`

and remove the section `UTExportedTypeDeclarations` which ensures that macOS does not associate all .love files with your app. Overall the changes should be something like this:

Original Info.plist	Modified Info.plist
... <key>CFBundleIdentifier</key> <string>**org.love2d.love**</string> ... <key>CFBundleName</key> <string>**LÖVE**</string> ... <key>NSPrincipalClass</key> <string>NSApplication</string> **<key>UTExportedTypeDeclarations</key>** **<array>** **...** **</array>** </dict> </plist>	... <key>CFBundleIdentifier</key> <string>**com.SuperCompany.SuperGame**</string> ... <key>CFBundleName</key> <string>**SuperGame**</string> ... <key>NSPrincipalClass</key> <string>NSApplication</string> </dict> </plist>

Official Distribution and the App Store

Compiling Mac apps requires a Mac running macOS with XCode installed (you can install XCode from the app store).

1. Download the love source code (https://github.com/love2d/love/releases/tag/11.3)

2. Download the required frameworks from here (http://love2d.org/sdk) and place them in `/Library/Frameworks/`.
3. Open the Xcode project found at `platform/xcode/love.xcodeproj`. You will build the `love-macosx` target as described below.
4. In XCode, Modify the macOS icons in `Images.xcassets`
5. In XCode, Update the `love-macosx.plist` (see instructions above)
6. Add your .love file to the "Copy Bundle Resources" Build Phase
    addLoveFileToXcodeBundleResources.png
7. Follow Apple's guide for distributing Mac apps (https://developer.apple.com/macos/distribution/)

Distribution for Linux

For Linux there is not yet a simple way to distribute your game. The general approach here is to point to the official LÖVE packages at https://www.love2d.org (https://www.love2d.org). Once the package is installed, the .love packages are usually automatically executed using the installed LÖVE package or using the command line:

```
love SuperGame.love
```

Make sure to include a `conf.lua` file with at least the LÖVE version used.

```
function love.conf(t)
    t.version = "11.4"
end
```

A **common mistake** is trying to fuse the love Linux binary with a love file:

```
cat /usr/bin/love SuperGame.love > SuperGame
chmod a+x ./SuperGame
```

Please keep in mind, while the binary `SuperGame` will run on your machine, and other Linux distros that have the same architecture and similar version libraries, there's a good chance that **it will not run on many other Linux distros** except when you're fusing with the official AppImage (see below).

Official AppImage

As of 11.4, the AppImage is relocatable. Meaning, if the AppImage contents are extracted then moved somewhere, then the executable inside it (`<root>/bin/love`) will just run without any problems. This means using Windows-style fusing using the binaries from the AppImage is now supported.

1. Download the AppImage from the Download Page (https://github.com/love2d/love/releases/latest)
2. Create a new directory, then run the `.AppImage` with `--appimage-extract` command-line parameter in that directory.
3. A new `squashfs-root` directory will be created. Fuse your game with `cat squashfs-root/bin/love SuperGame.love > squashfs-root/bin/SuperGame`
4. Set the execute flag using `chmod +x squashfs-root/bin/SuperGame`
5. Test out that the game is fused correctly by running it: `squashfs-root/bin/SuperGame`. If it works, delete the original `love` binary.

Then you can either just tar archive anything inside `squashfs-root` or repackage it into AppImage. If you choose the former,

1. Set the working directory first (`cd squashfs-root`)
2. create a new launcher script that runs `bin/SuperGame` relative to the current script location, then archive all of the directory contents with `tar czvf ../SuperGame.tar.gz *`

If you choose to repackage the game to AppImage,

1. Get `appimagetool` from here (https://github.com/AppImage/appimagetool/releases/tag/continuous).
2. Modify the `love.desktop` file to match your game description. Make sure to modify the `Exec=love %f` to `Exec=SuperGame %f`.
3. If you need to change the icon, place your PNG (SuperGame.png for example) or SVG (`SuperGame.svg`) icon beside `love.desktop` and modify the `Icon=love` to `Icon=SuperGame`. You can remove `love.svg`.
4. Repackage it to AppImage by running `appimagetool squashfs-root SuperGame.AppImage`.

Note: If you get an error when running the repackaged AppImage similar to `boot.lua: cannot load game at path ./tmp/.mount_SOMERANDOMCHARS/SuperGame` then you will also need to modify the AppRun file inside your squashfs-root folder so that the `exec` command points to your new fused binary - an example AppRun file would look like this:

```
#!/bin/sh
if [ -z "$APPDIR" ]; then
    APPDIR="$(dirname "$(readlink -f "$0")")"
fi
export LD_LIBRARY_PATH="$APPDIR/lib/:$LD_LIBRARY_PATH"
if [ -z "$XDG_DATA_DIRS" ]; then #unset or empty
    XDG_DATA_DIRS="/usr/local/share/:/usr/share/"
fi
export XDG_DATA_DIRS="$APPDIR/share/:$XDG_DATA_DIRS"
if [ -z "$LUA_PATH" ]; then
    LUA_PATH=";" # so ends with ;;
fi
# if user's LUA_PATH does not end with ;; then user doesn't want the default path ?
export LUA_PATH="$APPDIR/share/luajit-2.1.0-beta3/?.lua;$APPDIR/share/lua/5.1/?.lua;$LUA_PATH"
if [ -z "$LUA_CPATH" ]; then
    LUA_CPATH=";"
fi
export LUA_CPATH="$APPDIR/lib/lua/5.1/?.so;$LUA_CPATH"
# run the SuperGame bin
exec "$APPDIR/bin/SuperGame" "$@"
```

Linux

It has not been yet proven to work in all scenarios, but this a way of distributing your game for Linux.

1. Download the appropriate Linux version of love (with file extension .tar.gz), either x86_64, or i686. Download Page (https://github.com/love2d/love/releases)
2. Extract the `.tar.gz`. You can do this by either right clicking the file and pressing “extract” on most visual file managers, or extracting it via the command line:

   ```
   tar -xvzf love.tar.gz
   ```

3. Move your `game.love` file inside the extracted folder.
4. Edit the file called `love` by replacing "$@" with "${LOVE_LAUNCHER_LOCATION}/game.love”

Debian

There is a policy for including LÖVE games in the Debian archive. It can be followed to create a package for a game even if it is not intended to include it in the archive.

Packaging Love2d (https://w iki.debian.org/Games/Love2d)

AppImages

The AppImage format is a format for packaging applications in a way that allows them to run on a variety of different target systems (base operating systems, distributions) without further modification. The whole kit can be downloaded from here (https://github.com/probonopd/AppImageKit/).

In order to package games using AppImage, the libraries and dependencies required by love need to be gathered, which can be done easily using CARE (https://github.com/proot-me/proot/blob/master/doc/care/manual.rst#care) (available packaged in the Ubuntu repositories) by executing:

```
care -o love.tar.gz love
```

.

This program makes an archive, which contains the love executable and its dependencies and related files kept in the way of your system. Secondly, the game's .love file needs to be 'fused' by the means shown in the above section. Now, the files need to be placed in the following way in a new directory:

```
Application Directory/
├── <game name>.desktop  --> This file contains path to the program's icon file,
│                            the name of the program and the command to execute it.
├── <game name>.png      --> Your game's icon
└── usr/
    ├── bin/    --> Your fused binary needs to be placed here
    ├── lib/    --> Your libraries need to be placed here
    └── share/  --> Your (or LÖVE's) data files (and other things) need to be placed here, if any
```

Finally, the directory needs to be converted into an AppImage, using AppImageAssistant available in the kit. More information can be found in the AppImage documentation (https://docs.appimage.org/).

Android

Compiling from source

First, grab the source code either by downloading it from the Download Page (https://github.com/love2d/love/releases/latest) or by cloning the LÖVE Android project:

```
git clone --recurse-submodules https://github.com/love2d/love-android
```

(optionally with `--depth 1` to save download times)

Then, download the Android SDK (exactly API 31 for 11.4) and Android NDK (exactly version 21.3.6528147 or r21d for 11.4).

Set the `ANDROID_SDK_ROOT` to your Android SDK location then do either one of these to fuse your game:

1. Put your game files in `(project)/app/src/embed/assets` such that your main.lua path is `(project)/app/src/embed/assets/main.lua` (11.4 and later, recommended); or

2. Put your zipped `*.love` to `(project)/app/src/embed/assets/game.love`, noting the `game.love` filename (legacy, deprecated in next major update)

You may need to create the necessary directories.

Change the app launcher name (`android:label`) in `(project)/app/src/main/AndroidManifest.xml`. Change the package name (`applicationId`), version code (`versionCode`), and version string (`versionName`) in `(project)/app/build.gradle`.

For Linux or macOS systems it may be necessary set the executable permission:

```
chmod +x gradlew
```

And finally to generate the APK, do:

```
./gradlew assembleEmbedNoRecordRelease
```

The APK will be in `(project)/app/build/outputs/apk/embedNoRecord/release/app-embed-noRecord-release-unsigned.apk`

And if you need to generate AAB to be uploaded to Play Console, do:

```
./gradlew bundleEmbedNoRecordRelease
```

The AAB will be in `(project)/app/build/outputs/bundle/embedNoRecordRelease/app-embed-noRecord-release.aab`

For more information see the GitHub repository (https://github.com/love2d/love-android).

Using APKTool



APK is the format used to distribute your games independently for Android and is still supported by some of the smaller app stores. However, this method may not be supported by the Google Play Store.

The steps can be found here: Game Distribution/APKTool.

iOS

Basic instructions for iOS (without fusing) are available at Getting_Started#iOS.



Due to Apple restrictions, fusing for iOS is only possible under macOS!

To **fuse** your game into the LÖVE engine:

1. Get iOS source code from love2d.org (http://love2d.org).
2. Open platform/xcode/love.xcodeproj with Xcode.
3. Select love-ios target (not love-macosx)
4. Select **Build Phases** of the 'love-ios' target and add your 'game.love' file into **Copy Bundle Resources**.
5. Build the project.

6. Your game should appear on your iOS device as an app.

If you can't find the relevant XCode UI elements, you can refer to video tutorial on YouTube (https://youtu.be/MsYanw cU42E).

Distribution on the web

Publishing as a HTML5 or JavaScript game is possible thanks to Davidobot/love.js (https://github.com/Davidobot/love.js). Threads for example are not supported though and playback of course depends on the browser. Note that it may lag behind latest LÖVE version, check the repository for details.

The online tool LÖVE Web Builder (https://schellingb.github.io/LoveWebBuilder/) helps with using the **former** version of love.js. (May not be up to date.)

> *The available Love.js build is based on version 0.11.1 (a pre-release of the final 11.0) and not the latest version.*

There is a standalone version (https://github.com/2dengine/love.js/) of love.js that can load .love files without the intermediate "build" step.

Community Tools

There are a few tools developed by the community to help with game distribution.

- LÖVE Actions (https://github.com/love-actions) - Build & deploy game packages on ***MOST*** popular platforms with GitHub Actions. Supports Android, iOS, Linux, macOS, Windows. Doesn't support HTML5/WASM
- makelove (https://github.com/pfirsich/makelove), advanced multi-platform tool to fuse your game written in Python 3. Supports Windows and Linux with AppImage.
- boon (https://github.com/camchenry/boon) (forum (https://love2d.org/forums/viewtopic.php?t=86573)), a multi-platform, easy to use tool supporting Windows, macOS, Linux.
- Love Distribution Pack (https://love2d.org/forums/viewtopic.php?f=3&t=80565)[sic], a windows-only tool that can create .love files and .exe.
- love-dist (https://www.love2d.org/forums/viewtopic.php?f=5&t=30259), a windows-only tool that can LuaC your files and create an .exe.
- love-export (https://github.com/dmoa/love-export), a fast and simple command-line tool that builds binaries for you. Supports Windows, MacOS, and Linux.
- love-build (https://github.com/ellraiser/love-build), an app (built in LÖVE!) for quickly packaging LÖVE games for distribution.

You may also find other libraries under the distribution section on the Awesome Löve (https://github.com/love2d-community/awesome-love2d#distribution) list.

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

License

License · revision 31701 · last edited 2026-09-04

Summary

LÖVE (the software) is licensed under the terms of the ZLIB license. This is a very liberal license which permits almost anything.

The old, imported documentation is licensed under the FreeBSD Documentation license. This is similar to ZLIB, but meant for written documents as opposed to software.

ZLIB license

Copyright © 2006-2021 LÖVE Development Team

This software is provided 'as-is', without any express or implied warranty. In no event will the authors be held liable for any damages arising from the use of this software.

Permission is granted to anyone to use this software for any purpose, including commercial applications, and to alter it and redistribute it freely, subject to the following restrictions:

1. The origin of this software must not be misrepresented; you must not claim that you wrote the original software. If you use this software in a product, an acknowledgment in the product documentation would be appreciated but is not required.
2. Altered source versions must be plainly marked as such, and must not be misrepresented as being the original software.
3. This notice may not be removed or altered from any source distribution.

The FreeBSD Documentation license

Copyright © 2006-2010 LÖVE Development Team. All rights reserved.

Redistribution and use in source (XML) and compiled forms (HTML) with or without modification, are permitted provided that the following conditions are met:

1. Redistributions of source code (XML) must retain the above copyright notice, this list of conditions and the following disclaimer as the first lines of this file unmodified.
2. Redistributions in compiled form (HTML) must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.

THIS DOCUMENTATION IS PROVIDED BY THE LÖVE DEVELOPMENT TEAM "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE LÖVE DEVELOPMENT TEAM BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)

HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS DOCUMENTATION, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

Third Parties

LÖVE uses several third party libraries, each with their own license. A license.txt file is distributed with LÖVE and contains license information for each library. The latest revision of the license.txt file can be viewed on Github: https://github.com/love2d/love/blob/master/license.txt ↗

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Version History

Version History · revision 29431 · last edited 2025-01-14

Click on a version number to view the complete changelog of that version.

Version	Codename	Status
12.0	Bestest Friend	In development
11.5	Mysterious Mysteries	Released: 2023-12-03
11.4	Mysterious Mysteries	Released: 2022-01-02
11.3	Mysterious Mysteries	Released: 2019-10-27
11.2	Mysterious Mysteries	Released: 2018-11-25
11.1	Mysterious Mysteries	Released: 2018-04-15
11.0	Mysterious Mysteries	Released: 2018-04-01
0.10.2	Super Toast	Released: 2016-10-31
0.10.1	Super Toast	Released: 2016-02-14
0.10.0	Super Toast	Released: 2015-12-22
0.9.2	Baby Inspector	Released: 2015-02-14
0.9.1	Baby Inspector	Released: 2014-04-01
0.9.0	Baby Inspector	Released: 2013-12-13
0.8.0	Rubber Piggy	Released: 2012-04-02
0.7.2	Game Slave	Released: 2011-05-04
0.7.1	Game Slave	Released: 2011-02-14
0.7.0	Game Slave	Released: 2010-12-05
0.6.2	Jiggly Juice	Released: 2010-03-06
0.6.1	Jiggly Juice	Released: 2010-02-07
0.6.0	Jiggly Juice	Released: 2009-12-24
0.5.0	Salted Nuts	Released: 2009-01-02
0.4.0	Taco Beam	Released: 2008-08-29
0.3.2	Lemony Fresh	Released: 2008-07-04
0.3.1	Space Meat	Released: 2008-06-21
0.3.0	Mutant Vermin	Released: 2008-06-17
0.2.1	Impending Doom	Released: 2008-03-29
0.2.0	Mini-Moose	Released: 2008-02-06

0.1.1 (Initial Release)	Santa-Power	Released: 2008-01-13

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Guidelines

Guidelines · revision 2785 · last edited 2025-03-29

On this page

- Purpose
- Name Conflics
- Style
- Documentation
  - Modules
  - Types
  - Functions
  - Enums
  - Games
  - Libraries

All registered users can edit the wiki, but please follow these guidelines for consistency. If you think the guidelines should be changed, or should contain additional information, please don't hesitate to contact Rude.

Purpose

The main purposes of the wiki are as follows.

1. To document the LÖVE API, and provide tutorials.
2. To provide information about games and libraries using LÖVE.

That said, it is fully allowed to add information about other things which fall way outside these categories.

If you would like to make major changes in the wiki, like deploying a new structure for built-in types, for example; don't do it. You *must* talk to Rude first, otherwise there's a high chance your changes will be reverted.

Name Conflics

To avoid too much parenthesized URLs, documentation pages have plain titles without prefixes or suffixes. For instance, the type Image can be referred to externally as http://love2d.org/wiki/Image.

This may occasionally lead to name conflicts, in which case the documentation page will always win. If you create a game called *Awesome*, and LÖVE later decides to expose a type called *Awesome* (not completely unlikely), then your game will be moved to another title.

Style

If you're talking about types built into LÖVE, link to them. For instance, Image is an object which can be displayed on screen.

When referring to variables and code inside a regular sentence, use the <code> tag to improve the reading experience. For instance: when you add `foo` and `bar`, the result is `foobar`.

Documentation

Documentation pages *must* follow the same structure if the documentation is to be usable.

All documentation pages must have a semantic link to their parent in their *See Also* section. Without this, the item will not appear in the automatic lists throughout the wiki. A semantic link is added like this for a function, type or enum in love.audio:

```
[[parent::love.audio]]
```

All documentation pages must also have a semantic, 'list-friendly' description. This is done by adding the following to the bottom of the page:

```
{{#set:Description=Draws an [[Image]] on screen.}}
```

Modules

The following 'level 2' headers are allowed, but should only be used if they contain anything.

- Types
- Functions
- Enums
- Notes
- Examples
- See Also

All modules must be assiciated with Category:Modules.

Types

The following 'level 2' headers are allowed, but again, don't use then and leave them empty.

- Functions
- Enums
- Supertypes
- Subtypes
- Notes
- Examples
- See Also

All types must be assiciated with Category:Types.

Functions

For functions, all headers are *required*, except *Notes*, *Examples* and *See Also*. This means that if a function returns nothing, it should explicitly say that it returns nothing. The same goes for arguments.

Similarly, if there are no constructors for a certain type (e.g. Contact), this should be explicitly mentioned.

The outer items in the list below are 'level 2' headers, and the inner ones are 'level 3'.

- Constructors
- Function
    - Synopsis
    - Arguments
    - Returns
- Notes
- Examples
- See Also

If a function has overloads, repeat the 'Function' header for each overload. This is not required for trailing optional arguments; just indicate the default value in parentheses instead.

Arguments and return values should be presented as a definition list, using Template:param.

All functions must be assiciated with Category:Functions.

Enums

For enums, these headers are allowed:

- Constants
- Notes
- See Also

The constants should be listed as a definition list.

All enums must be associated with Category:Enums.

Games

For games, you can choose your own structure, but you should always add some semantic properties to the page to make it appear correctly in the list of games:

```
{{#set:Name=NoGame}} (Should be same as page title)
{{#set:Author=User:Rude}}
{{#set:Genre=What kind of game your game is}}
{{#set:LOVE Version=0.6.1}}
{{#set:Description=A short description of your game.}}
{{#set:Screenshot=File:ScreenshotURL.png}}
```

Screenshots will automatically be resized to fit a box of 161x100 pixels.

Also, put your game in Category:Games to make it appear automatically in the Games page. To put your game in the correct category, add this line at the end of the page.

```
[[Category:Games]]
```

It is also recommended that you provide screenshots of your game.

Libraries

No strict rules, but as for games, you should add these semantic properties:

```
{{#set:LOVE Version=0.6.1}}
{{#set:Description=A short description of your library.}}
```

Put your library in Category:Libraries to make it appear automatically in on the Libraries page. To put your library in the correct category, add this line at the end of the page.

```
[[Category:Libraries]]
```

Category:Tutorials

Category:Tutorials · revision 29458 · last edited 2025-09-30

Beginner tutorials

Title	LÖVE version	Format	Creator	Description
How to LÖVE (http://sheepolution.com/learn/book/contents)	11.3	Text	Sheepolution	Comprehensive tutorial teaching various concepts that a beginner needs. Starts with explaining what variables are and ends with how to make a platformer.
Love2D Lua Tests (https://github.com/darkfrei/love2d-lua-tests)	11.5	Code examples	darkfrei	Plenty of code examples that work as quick tutorials for various game-related issues and development: graphics, vectors, polygons, interaction, collisions, raycasting, tiles, Voronoi graphs, meshes, menus, Android development and much more...
LÖVE 2D En Español (https://love2d.org/wiki/Category:Tutorials_(Espa%C3%B1ol))	11.3	Video	Rusbelito	En este Tutorial/curso Aprenderás desde lo mas básico como ¿Qué Es una Variable? como AL Final Crear Tu Propia Idea Para Un Videojuego En 2D
learn2love (https://rvagamejams.com/learn2love/)	11.3	Text	RVA Game Jams	This book teaches programming from the ground up in the context of Lua and LÖVE. It teaches basic computer science and software building skills along the way.
Simple Game Tutorial (https://simplegametutorials.github.io/)	11.3	Text	Santos	A list of short tutorials that explain how to make all kinds of games like Snake, Tetris and Flappy Bird.
Cameras (https://ebens.me/post/cameras-in-love2d-part-1-the-basics)	11.3	Text	Michael Ebens	A 3-part tutorial covering the use of Cameras
CS50's Introduction to Game Development (https://www.youtube.com/watch?v=GfwpRU0cT10)	0.10.2	Video	CS50 (Harvard University)	A lecture on LÖVE where it teaches how to make Pong, explaining everything along the way.
CS50's Falling in LÖVE with Lua (https://youtu.be/JJ_ypbTQLws?t=598)	?	Video	CS50 (Harvard University)	A lecture on LÖVE where it teaches how to make Super Mario, explaining everything along the way.

How to LÖVE (https://www.youtube.com/watch?v=TUPbSJ5wF0k&list=PLM5EvDHhpyTcThnWfeP1459KelemQOBdG)	0.10.1	Video	Sheepolution	The video version of How to LÖVE. Starts with the basics and ends with how to make Pong.
Game Development For Complete Beginners (https://www.youtube.com/watch?v=HYYsedq2Ng4)	0.10.0	Video	GamesFromScratch	A tutorial on game development in general that uses LÖVE.
Making an Arkanoid-type Game with LÖVE Framework (https://github.com/noooway/love2d_arkanoid_tutorial/wiki)	?	Text	Noooway	Make an Arkanoid-type Game from scratch.
Your First LÖVE Game in 200 Lines (http://osmstudios.com/tutorials/your-first-love2d-game-in-200-lines-part-1-of-3)	0.9.1	Text	OSMstudios	A tutorial where you make a small shooter game from scratch.
Love Tile Tutorial (https://github.com/kikito/love-tile-tutorial/wiki)	0.6.0	Text	Kikito	Teaches how to make a tile-based level with a moving character, collisions and scrolling, starting from scratch.

All tutorials on the Wiki

- GLES Shader Testing in Desktop
- In-Memory Audio Streaming
- examples.love

- A Guide to Getting Started with Love2D
- A Guide to Getting Started with Love2D (Dansk)
- A Guide to Getting Started with Love2D (Português)
- A Guide to Getting Started with Love2D (Русский)
- Advanced Tiled Loader
- Advanced Tiled Loader (Français)
- Advanced Tiled Loader (日本語)
- Animation
- Animation (Français)
- Animation (日本語)
- Audio
- Audio (Deutsch)
- Audio (Español)
- Audio (Français)
- Audio (Polski)
- Audio (Português)

- Audio (Tiếng Việt)
- Audio (Русский)
- Audio (日本語)
- Audio (简体中文)
- Baseline 2D Platformer
- Baseline 2D Platformer (日本語)
- Callback Functions
- Callback Functions (Deutsch)
- Callback Functions (Español)
- Callback Functions (Français)
- Callback Functions (Italiano)
- Callback Functions (Polski)
- Callback Functions (Português)
- Callback Functions (Svenska)
- Callback Functions (Tiếng Việt)
- Callback Functions (Русский)
- Callback Functions (Українська)
- Callback Functions (日本語)
- Callback Functions (简体中文)
- Callback Functions (한국어)
- Cameras
- Cameras:camera
- Cameras:camera (Polski)
- Cameras (Français)
- Cameras (Polski)
- Cameras (简体中文)
- Custom Cursors
- Custom Cursors (日本語)
- Draw Origins
- Drawing Order
- Drawing Order (Français)
- Drawing Order (Tiếng Việt)
- Drawing Order (Русский)
- Drawing Order (日本語)
- Efficient Tile-based Scrolling
- Efficient Tile-based Scrolling (Français)
- Efficient Tile-based Scrolling (Português)
- Efficient Tile-based Scrolling (日本語)
- Fine Tile-based Scrolling
- Fine Tile-based Scrolling (日本語)
- Fire Toward Mouse
- Fire Toward Mouse (日本語)
- Fonts and Text
- Fonts and Text (Deutsch)
- Fonts and Text (Français)
- Fonts and Text (Русский)
- Fonts and Text (日本語)

- Fonts and Text (简体中文)
- Graphic Tranformations
- Graphic Transformations
- Graphic Transformations (日本語)
- Gridlocked Player
- Gridlocked Player (日本語)
- Hamster Ball
- Hamster Ball (Deutsch)
- Hamster Ball (Español)
- Hamster Ball (Nederlands)
- Hamster Ball (Tiếng Việt)
- Hamster Ball (日本語)
- Introduction to Shaders
- Introduction to Shaders (日本語)
- Isometric Graphics
- Isometric Graphics (Русский)
- Isometric Graphics (日本語)
- Mouse Dragging (Русский)
- Mouse Dragging in Love2D
- Networking with UDP
- Networking with UDP-TheClient
- Networking with UDP-TheClient (日本語)
- Networking with UDP-TheServer
- Networking with UDP-TheServer (日本語)
- Networking with UDP (Français)
- Networking with UDP (日本語)
- Object Follows Another Object
- Object Follows Another Object (日本語)
- Object Follows Mouse
- Object Follows Mouse (日本語)
- Physics
- PhysicsCollisionCallbacks
- PhysicsCollisionCallbacks (Русский)
- PhysicsCollisionCallbacks (日本語)
- PhysicsCollisionCallbacks (简体中文)
- PhysicsDrawing
- PhysicsDrawing (日本語)
- Physics (Français)
- Physics (Tiếng Việt)
- Physics (Česky)
- Physics (Русский)
- Physics (日本語)
- Physics (简体中文)
- Platformer Jumping
- Setting up PyCharm
- Setting up PyCharm (日本語)
- TestingPage

- Tile-based Scrolling
- Tile-based Scrolling (日本語)
- Using Input
- Using Input (Polski)
- Using Input (Português)
- Using Input (日本語)

Other languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info